File and Directory Operations

File and directory operations are essential for handling stored data. Python provides built-in functions through the `open()`, `os`, and `shutil` modules to perform these tasks efficiently.

1. File Operations

1.1 Opening a File

Python uses the `open()` function to open a file in different modes.

```python
file = open("example.txt", "r")  # Opens file in read mode
```

Syntax:

```python
open(filename, mode)
```

Modes:

Mode	Description
`'r'`	Read (default mode)
`'w'`	Write (overwrites existing content)
`'a'`	Append (adds to the end of the file)
`'b'`	Binary mode (e.g., for images, PDFs)
`'+'`	Read and Write

1.2 Checking if a File Exists Before Opening

Use `os.path.exists()` to check if a file exists.

```python
import os
if os.path.exists("example.txt"):
    file = open("example.txt", "r")
    print("File opened successfully.")
else:
    print("File does not exist.")
```

2. Reading a File

```python
file = open("example.txt", "r")
content = file.read()       # Reads the entire content
lines = file.readlines()    # Reads all lines into a list
line = file.readline()      # Reads a single line
print(content)
file.close()
```

Best Practice: Use the `with` statement to automatically close the file.

```python
with open("example.txt", "r") as file:
    content = file.read()
    print(content)
```

3. Writing to a File

```python
with open("example.txt", "w") as file:
    file.write("Hello, World!")  # Overwrites existing content
```

> **Note:** `'w'` mode erases previous content.

4. Appending to a File

```python
with open("example.txt", "a") as file:
    file.write("\nNew Line Added.")  # Adds new content at the
end
```

5. File Path Operations

5.1 Using `os.path.join()` for Cross-Platform Compatibility

Use `os.path.join()` to construct file paths that work across different operating systems.

```python
import os
file_path = os.path.join("folder", "example.txt")
with open(file_path, "r") as file:
    print(file.read())
```

5.2 Getting File Information

```python
import os
file_path = "example.txt"
if os.path.exists(file_path):
    print("File Size:", os.path.getsize(file_path), "bytes")
    print("Absolute Path:", os.path.abspath(file_path))
else:
    print("File does not exist")
```

6. Directory Operations in Python

Python's `os` and `shutil` modules provide functions to manage directories.

6.1 Creating a Directory

```python
import os
os.mkdir("new_folder")  # Creates a single directory
```

Creating Nested Directories

```python
os.makedirs("parent_folder/child_folder")
```

6.2 Checking if a Directory Exists

```python
if not os.path.exists("new_folder"):
    os.mkdir("new_folder")
```

6.3 Removing a Directory

```python
import os
os.rmdir("new_folder")  # Removes an empty directory
```

Removing a Non-Empty Directory

```python
import shutil
shutil.rmtree("parent_folder")  # Deletes folder and all its contents
```

6.4 Listing Files in a Directory

```python
files = os.listdir(".")  # Lists all files and folders in the current directory
for file in files:
    print(file)
```

6.5 Changing the Current Working Directory

```python
import os
os.chdir("path/to/directory")  # Changes to the specified directory
print("Current Directory:", os.getcwd())  # Prints the current working directory
```

7. Real-World Example: Logging System

```python
def log_activity(activity):
    with open("log.txt", "a") as log_file:
        log_file.write(activity + "\n")

log_activity("User logged in.")
log_activity("User uploaded a file.")
```

Purpose: Appends each user activity to `log.txt`, useful for monitoring system actions.

8. Real-World Example: Organizing Files by Type

Python

```
import os
import shutil

def organize_files(folder):
    if not os.path.exists(os.path.join(folder, "Images")):
        os.mkdir(os.path.join(folder, "Images"))
    if not os.path.exists(os.path.join(folder, "Documents")):
        os.mkdir(os.path.join(folder, "Documents"))

    for file in os.listdir(folder):
        file_path = os.path.join(folder, file)
        if os.path.isfile(file_path):
            if file.endswith(".jpg"):
                shutil.move(file_path, os.path.join(folder,
"Images", file))
            elif file.endswith(".pdf"):
                shutil.move(file_path, os.path.join(folder,
"Documents", file))

organize_files("Downloads")
```

Purpose: Automatically organizes files into different folders based on their extensions.

9. Summary of Key Functions

Operation	Function	Module
Open File	`open()`	Built-in
Read File	`read()`, `readlines()`, `readline()`	Built-in
Write/Append File	`write()`	Built-in
Close File	`close()`	Built-in
Check if File Exists	`os.path.exists()`	`os`
Get File Size	`os.path.getsize()`	`os`
Get Absolute Path	`os.path.abspath()`	`os`
Create Directory	`os.mkdir()`, `os.makedirs()`	`os`
Remove Directory	`os.rmdir()`, `shutil.rmtree()`	`os`, `shutil`
List Directory Contents	`os.listdir()`	`os`
Change Directory	`os.chdir()`	`os`
Join File Path	`os.path.join()`	`os`